

Universidad ORT Uruguay

Facultad de Ingeniería

Bernard Wand Polak

Diseño de Aplicaciones 2

Evidencia de TDD

Joaquin Rodríguez (231355)

Agustín Madrid (270635)

Juan Diego Meneses (224304)

2024

Índice

Descripción general del trabajo	2
Errores conocidos y Deuda técnica	2
Diagrama general de paquetes	4
Diagrama de componentes	5
Modelo de tablas de la base de Datos	10
Diagramas de interacción	11
Post de un admin:	11
Post de un Login	11
Decisiones de Diseño	12
Cómo funcionan los roles y los permisos	12
Cómo funcionan los Home Devices	13
La creación de la clase MemberSettings y para que funciona	14
Manejo de Excepciones y Exceptionfilter	14
Hacer DevicesTypes Enum	15
Implementación de la Seed Data	15

Descripción general del trabajo

El objetivo de nuestra solución es proporcionar una API que centralice y gestione diversos dispositivos inteligentes en el hogar, creando un entorno integrado y automatizado. Los dispositivos inteligentes incluyen cámaras de seguridad y sensores de ventana. Nuestra plataforma permite que múltiples usuarios con diferentes roles, como administradores, dueños de empresas y dueños de hogares, puedan interactuar de manera eficiente y segura con estos dispositivos.

Cada rol tiene permisos específicos que determinan qué acciones pueden realizar. Los administradores pueden gestionar las cuentas de los usuarios, crear nuevas cuentas de dueños de empresa, listar todas las cuentas registradas, así como visualizar empresas que distribuyen dispositivos. Los dueños de empresas, por otro lado, pueden registrar empresas y dispositivos como cámaras de seguridad y sensores, configurando detalles específicos como la compatibilidad con detección de movimiento y de personas.

Finalmente, los dueños de hogares pueden gestionar sus dispositivos, configurar notificaciones para recibir alertas sobre eventos importantes (como la detección de movimiento en una cámara) y añadir miembros a su hogar para que colaboren en la administración de los dispositivos. Esta estructura asegura que las operaciones relacionadas con la seguridad y la administración del hogar inteligente se centralizan y sean fácilmente accesibles a través de una única plataforma.

Decidimos hacer un approach Top-Down a nivel de TDD, moqueando los componentes de más bajo nivel a medida que íbamos implementando las interfaces que necesitábamos. Esto nos permitió desarrollar primero las funcionalidades de la capa de presentación y luego trabajar en la lógica de negocios y la persistencia de datos.

Para organizar el trabajo creamos un tablero de JIRA con tickets y cada uno de los integrantes del equipo fue asignándose a medida que implementamos la aplicación. Este consistía de 4 columnas: la de To Do, In Progress, In Review y Done.

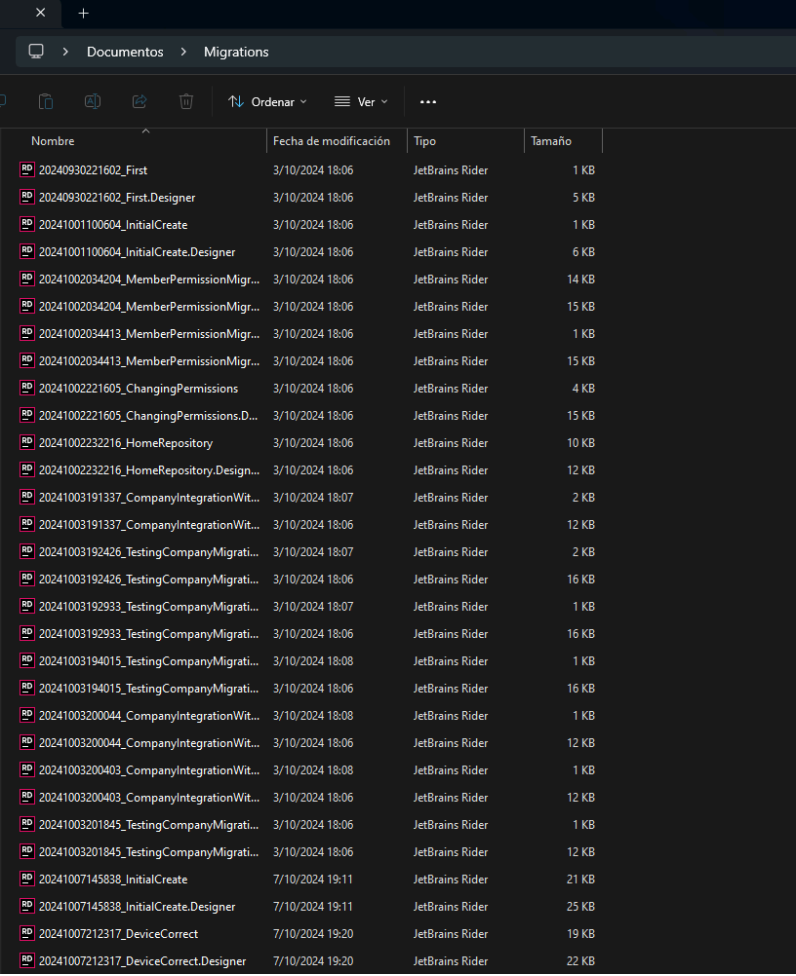
Por lo general siempre que se ponía In Review un ticket era para que un compañero revisará de manera detallada el código. Utilizamos el code Analysis proporcionado en clase y nombres de clases y métodos explícitos para mantener el clean Code en todo momento.

Errores conocidos y Deuda técnica

A groso modo todas las funcionalidades requeridas están implementadas y funcionando correctamente. Sin embargo sabemos que nuestro diseño a nivel de paquetes puede mejorarse ya que tenemos algunos paquetes que los podríamos poner juntos un ejemplo serían el de Dominio y el de BusinessLogic los cuales están muy acoplados y deberían de ir en un paquete juntos.

Otro error conocido es nuestra utilización de entity Framework Core y las migraciones. Nuestro equipo tuvo ciertas dificultades con las migraciones y la forma que encontramos para solucionarlo fue eliminar las migraciones y volver a crear la base de datos. Sabemos que en un entorno de producción eso no sería posible y que si la cantidad de datos fuera mayor también sería un

problema. En el repositorio pondremos evidencia de que estas migraciones existieron realmente en una carpeta llamada Testimonio de migraciones.



Nombre	Fecha de modificación	Tipo	Tamaño
20240930221602_First	3/10/2024 18:06	JetBrains Rider	1 KB
20240930221602_First.Designer	3/10/2024 18:06	JetBrains Rider	5 KB
20241001100604_InitialCreate	3/10/2024 18:06	JetBrains Rider	1 KB
20241001100604_InitialCreate.Designer	3/10/2024 18:06	JetBrains Rider	6 KB
20241002034204_MemberPermissionMigr...	3/10/2024 18:06	JetBrains Rider	14 KB
20241002034204_MemberPermissionMigr...	3/10/2024 18:06	JetBrains Rider	15 KB
20241002034413_MemberPermissionMigr...	3/10/2024 18:06	JetBrains Rider	1 KB
20241002034413_MemberPermissionMigr...	3/10/2024 18:06	JetBrains Rider	15 KB
20241002221605_ChangingPermissions	3/10/2024 18:06	JetBrains Rider	4 KB
20241002221605_ChangingPermissions.D...	3/10/2024 18:06	JetBrains Rider	15 KB
20241002232216_HomeRepository	3/10/2024 18:06	JetBrains Rider	10 KB
20241002232216_HomeRepository.Design...	3/10/2024 18:06	JetBrains Rider	12 KB
20241003191337_CompanyIntegrationWit...	3/10/2024 18:07	JetBrains Rider	2 KB
20241003191337_CompanyIntegrationWit...	3/10/2024 18:06	JetBrains Rider	12 KB
20241003192426_TestingCompanyMigrati...	3/10/2024 18:07	JetBrains Rider	2 KB
20241003192426_TestingCompanyMigrati...	3/10/2024 18:06	JetBrains Rider	16 KB
20241003192933_TestingCompanyMigrati...	3/10/2024 18:07	JetBrains Rider	1 KB
20241003192933_TestingCompanyMigrati...	3/10/2024 18:06	JetBrains Rider	16 KB
20241003194015_TestingCompanyMigrati...	3/10/2024 18:08	JetBrains Rider	1 KB
20241003194015_TestingCompanyMigrati...	3/10/2024 18:06	JetBrains Rider	16 KB
20241003200044_CompanyIntegrationWit...	3/10/2024 18:08	JetBrains Rider	1 KB
20241003200044_CompanyIntegrationWit...	3/10/2024 18:06	JetBrains Rider	12 KB
20241003200403_CompanyIntegrationWit...	3/10/2024 18:08	JetBrains Rider	1 KB
20241003200403_CompanyIntegrationWit...	3/10/2024 18:06	JetBrains Rider	12 KB
20241003201845_TestingCompanyMigrati...	3/10/2024 18:06	JetBrains Rider	1 KB
20241003201845_TestingCompanyMigrati...	3/10/2024 18:06	JetBrains Rider	12 KB
20241007145838_InitialCreate	7/10/2024 19:11	JetBrains Rider	21 KB
20241007145838_InitialCreate.Designer	7/10/2024 19:11	JetBrains Rider	25 KB
20241007212317_DeviceCorrect	7/10/2024 19:20	JetBrains Rider	19 KB
20241007212317_DeviceCorrect.Designer	7/10/2024 19:20	JetBrains Rider	22 KB

Otra posible mejora sería que los endpoints de tipo GET en nuestra aplicación si no encuentran nada devuelva una lista vacía con un 200. Esto lo podríamos mejorar haciendo que si la lista está vacía de responder con un 204 NoContent en lugar del 200.

Otra posible mejora es crear un paquete exclusivo para las excepciones en lugar de tenerlas en los paquetes en donde las usamos para lanzar los errores. Esto ayudaría a continuar con el SDP a nivel de paquetes.

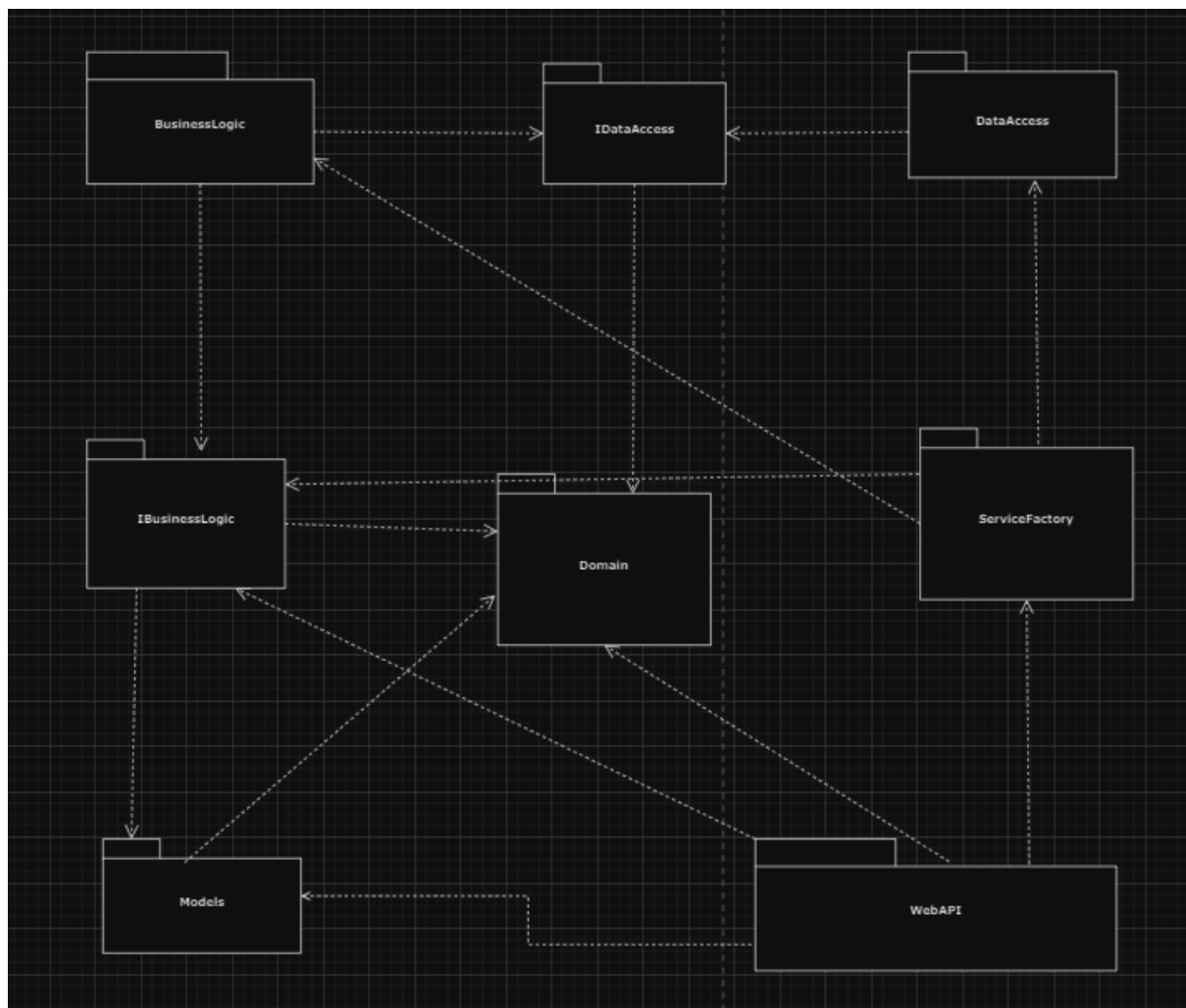
Un chequeo que pasamos por alto que vamos a implementar en el siguiente obligatorio es el chequeo de formato url para las fotos.

Como mala práctica el Delete de Admin y de User devuelven todo el usuario incluyendo la contraseña por más que ya no sea válida consideramos que se debería de cambiar por un User response;

Otro error a corregir que encontramos fue que se da un token de Session inexistente o sea que sea de tipo token pero no este en la session repository la aplicación devuelve un 500 internal Error en lugar de un 403 Unauthenticated.

Diagrama general de paquetes

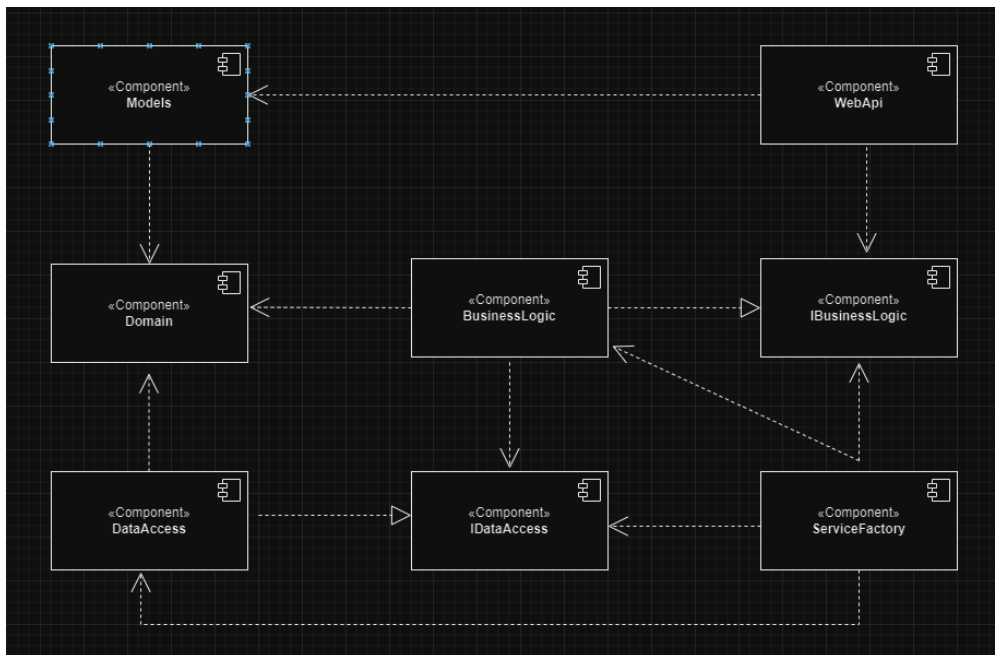
En este diagrama se puede apreciar los distintos paquetes que existen en la solución del obligatorio, sin contar los paquetes para testing. Estos paquetes son: BusinessLogic, IBusinessLogic, Domain, DataAccess, IDataAccess, ServiceFactory, Models, y WebAPI. Los paquetes de test se dividieron en 4 paquetes temáticos (Controllers, Logic, repository y Filters).



https://drive.google.com/file/d/1tEAzdd5QpVxYudPttGb4O7s8a_2Vt9Vg/view?usp=sharing

Diagrama de componentes

A nivel del modelo 4 +1 en la vista lógica ponemos este diagrama que expone todos los componentes necesarios en aplicación.

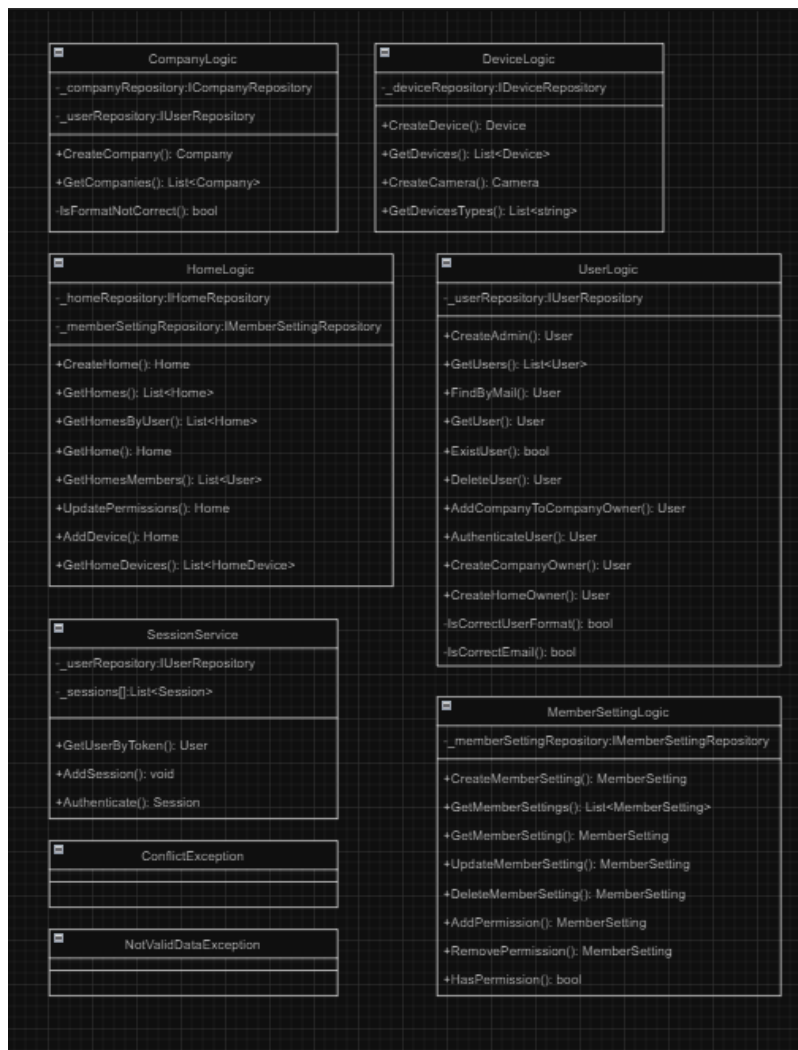


<https://drive.google.com/file/d/18ORSYRWQLoSw7SHVDgMcOiLRexIUYPXu/view?usp=sharing>

Paquetes

BusinessLogic

Contiene la lógica del negocio para la solución, implementando las operaciones de IBusinessLogic. Además en este paquete mantenemos 2 de las 3 excepciones que utilizamos en nuestro obligatorio estas heredan de exception.



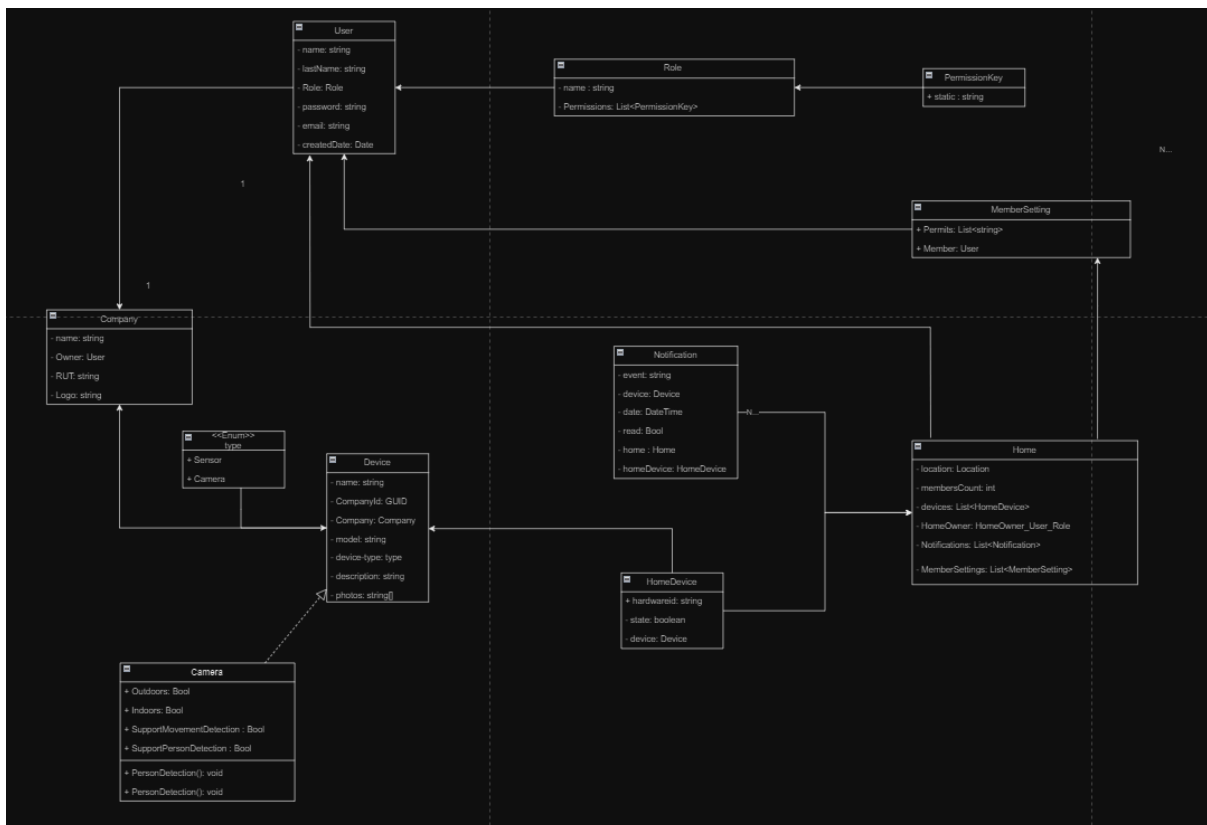
IBusinessLogic

Contiene las interfaces utilizadas para las distintas operaciones que hay en BusinessLogic. Son importantes debido a la inyección de dependencias la cual nos permite invertir las dependencias para mantener Liskov.



Domain

Todas las entidades utilizadas para realizar el proyecto.

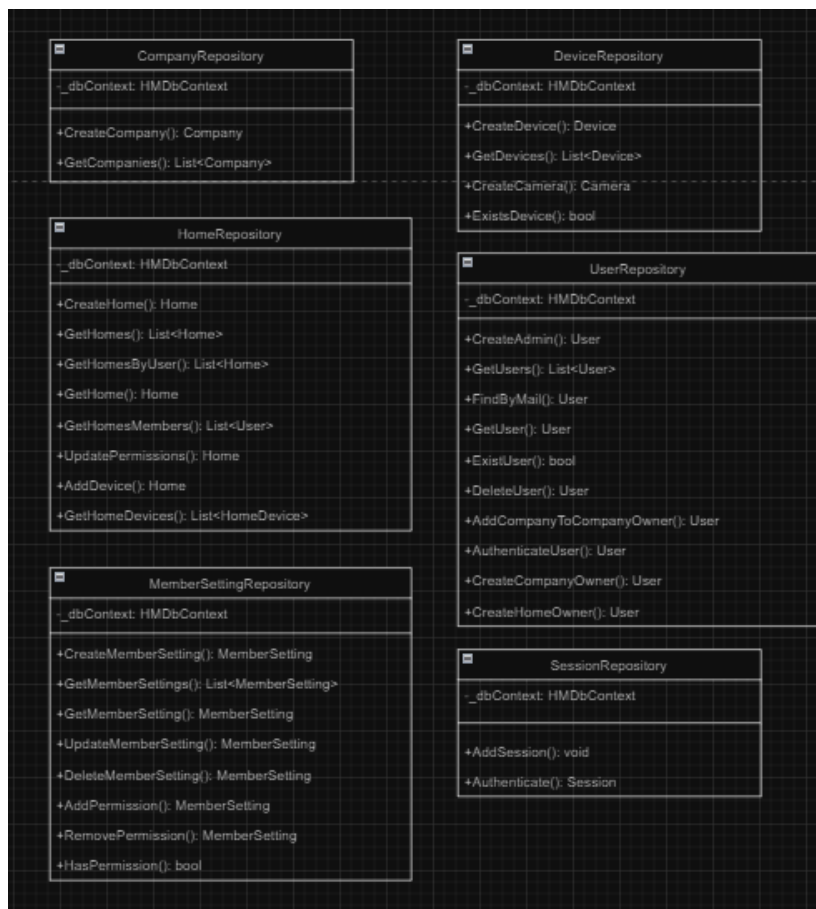


https://drive.google.com/file/d/1WKss8Ek_106_NkFe0nXOb13eZ-MDle-4/view?usp=sharing

En este paquete mantenemos todas las entidades necesarias para la implementación de nuestra solución. En este mantenemos las relaciones que las detallaremos en la sección de decisiones de diseño. Los Devices types los creamos como un enum porque la letra explícita que no se cambian de manera habitual.

DataAccess

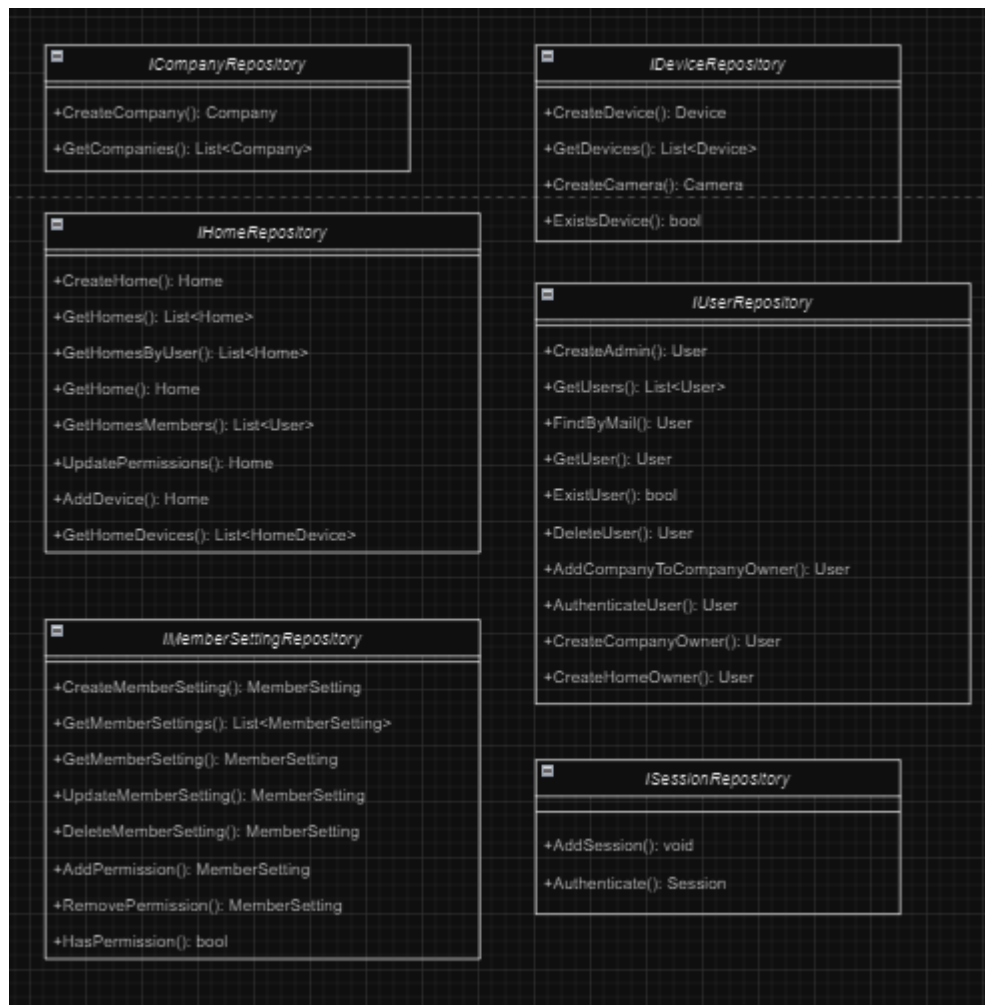
Implementa las operaciones de la interfaz IDataAccess, usando EntityFramework.



En esta tenemos nuestra otra excepción llamada EntityNotFoundException la cual usamos para cuando las requests más complejas les falta información en la base de datos (Un ejemplo sería el caso de un CompanyOwner incompleto, el cuál todavía no registro su empresa quiere crear un device).

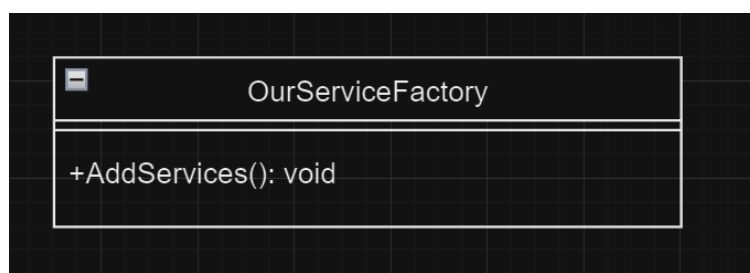
IDataAccess

Interfaz con las operaciones utilizadas para el acceso a datos.



ServiceFactory

El paquete ServiceFactory se encarga de la inyección de dependencias y la conexión a la base de datos. Decidimos separar este paquete del de WebApi por una cuestión de que cada paquete tenga una sola responsabilidad clara, queriendo preservar a la Api solo con los Controllers y los filters.



WebAPI

Contiene los Controllers, donde se encuentran las clases encargadas de recibir mensajes HTTP y responderlos. También contiene los Filters. Decidimos separar en 3 controllers distintos los usuarios separados por su rol cada controller mantiene las funcionalidades de cada uno. Además tenemos otros controllers para otras las funcionalidades relacionadas relacionadas con otras entidades como Company, Devices o Home. Además el controller de login que nos devuelve un token necesario para las autenticaciones y autorizaciones de todos los endpoints excepto el de CreateHomeOwner.

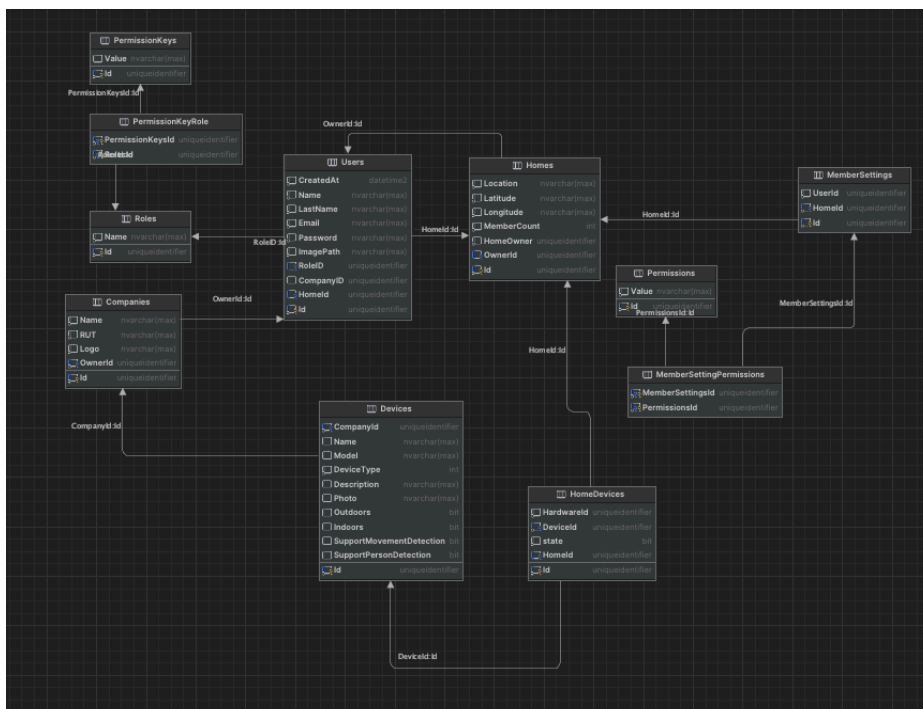
Descripción de jerarquías de herencias utilizadas

La clase Camera hereda de la clase Device, por lo tanto tiene todos los atributos de Device además de los suyos propios. Esto nos sirve para poder tener todos los devices en el mismo repositorio esto lo configuramos en el Context de nuestra aplicación. Esto se llama es una estrategia que se llama TPH (tables per Hierarchy). Que no permite guardar en una misma tabla los objetos padre e hijo.

```
modelBuilder.Entity<Device>() // EntityTypeBuilder<Device>
    .HasDiscriminator<DeviceType>(name: "DeviceType")
    .HasValue<Device>(DeviceType.Sensor)
    .HasValue<Camera>(DeviceType.Camera);
```

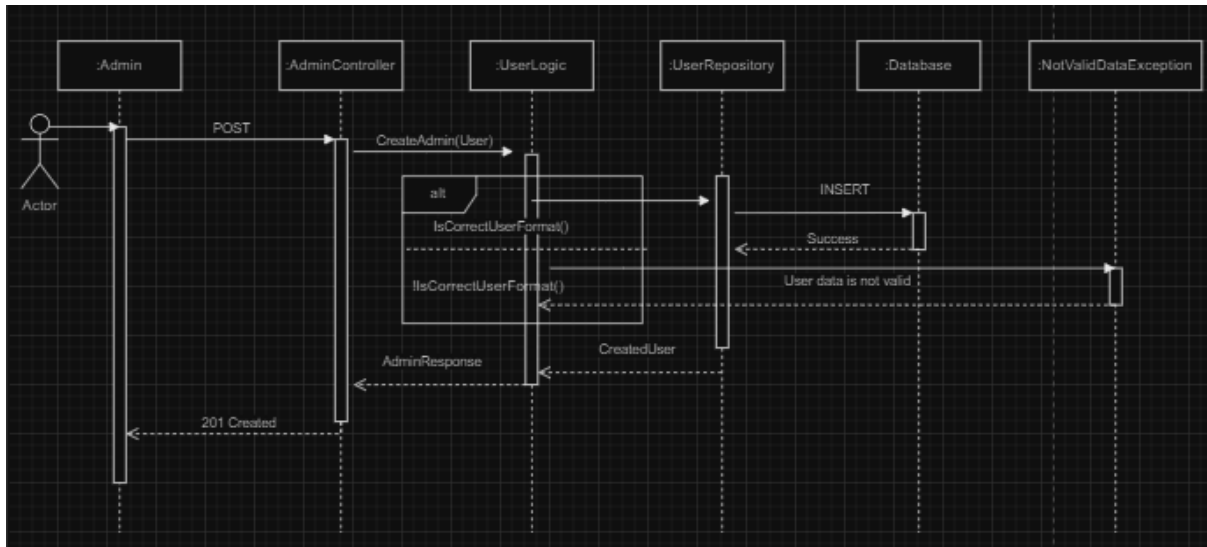
Modelo de tablas de la base de Datos

Después de muchas iteraciones y cambios nuestro modelo final de tablas es el siguiente:



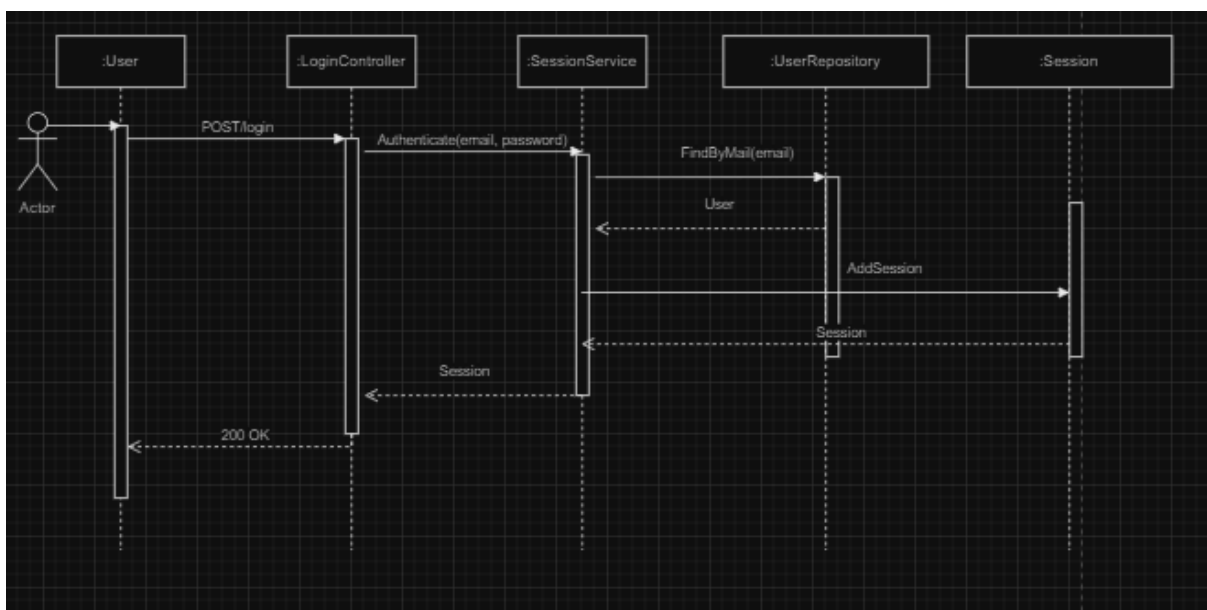
Diagramas de interacción

Post de un admin:



En este diagrama, podemos ver como funciona hacer Post de un Admin. Este diagrama es un ejemplo de cómo la aplicación pasa por las 3 layers desde un controller que recibe la request pasando por la lógica para después llegar al repositorio. Todo el resto de funciones en los endpoints funcionan de manera similar a excepción del login.

Post de un Login



<https://drive.google.com/file/d/1ZW3Q5gTVIXDM6wwp7tIYRiupZo-uOSxD/view?usp=sharing>

En este diagrama, podemos ver como es la secuencia de la función Login.

Decisiones de Diseño

Cómo funcionan los roles y los permisos

Para empezar a explicar como funcionan los roles y los permisos, debemos hacer una pequeña introducción a los elementos que usamos para diseñar esta solución.

Por un lado tenemos a los Roles o como está definido en nuestro dominio : **Role**.

Esta clase es la encargada de tener el rol, su nombre, y los **PermissionKey** correspondientes a cada rol. Podemos ver que dentro de rol, esta lista de **PermissionKeys** es virtual. Cuando una propiedad de navegación se declara como virtual, **EF Core** puede interceptar las llamadas a esa propiedad. Esto permite que la propiedad se cargue automáticamente desde la base de datos en el momento en que se accede, en lugar de cargarse junto con la entidad principal.

Por otro lado **PermissionKey** es una clase que almacena los diferentes permisos para cada rol.

Este conjunto de ideas, nos permitió que a la hora de utilizar filtro de autorización, podamos limitar las funcionalidades a las que cada rol puede acceder.

Veamos un ejemplo, si un Admin quiere acceder a los datos de los usuarios. Primero se chequea que esté logueado. Luego se chequea el rol con su nombre y se busca en sus permisos el permiso que el controller le está pidiendo, en este caso por ejemplo sería *"CanManageUsers"*.

Ventajas de Utilizar Roles y Permisos

Control Granular de Acceso: Permite definir con precisión qué acciones puede realizar cada rol dentro del sistema. Esto asegura que los usuarios solo tengan acceso a las funcionalidades necesarias para su trabajo, mejorando la seguridad.

Facilidad de Mantenimiento: Al centralizar los permisos en la clase **PermissionKey**, es más fácil realizar cambios. Si se necesita modificar un permiso o agregar uno nuevo, solo es necesario actualizar la clase **PermissionKey** o la lista de permisos en el rol correspondiente, sin tener que modificar la lógica de acceso en múltiples partes del código.

Escalabilidad: Este enfoque permite escalar el sistema fácilmente. A medida que se añaden nuevos roles o permisos, se pueden gestionar sin complicaciones adicionales. Por ejemplo, si se introduce un nuevo rol "Editor", simplemente se pueden definir los permisos correspondientes sin alterar la estructura existente.

Auditoría y Seguimiento: Al tener roles y permisos bien definidos, es más sencillo llevar a cabo auditorías de seguridad y seguimiento de quién accede a qué funcionalidades. Esto es fundamental para mantener la conformidad y detectar cualquier acceso no autorizado.

Cómo funcionan los Home Devices

Los **HomeDevices**, son una clase intermedia que se genera al relacionar un Device con una Home. Básicamente al agregar un dispositivo a una casa, creamos un **HomeDevice** que nos viene con hardware id y otras características pedidas específicamente para la asignación de un dispositivo a un hogar.

Creación de HomeDevices

Cuando un usuario decide agregar un dispositivo a una casa, se genera automáticamente un objeto de tipo **HomeDevice**. Este objeto actúa como un enlace entre el dispositivo y el hogar, asegurando que cada dispositivo esté correctamente asociado a un entorno específico. Durante este proceso de creación, el **HomeDevice** se configura con varias características importantes:

Hardware ID: Este identificador único permite rastrear y gestionar el dispositivo en la red del hogar. Es esencial para realizar operaciones como la activación, desactivación o monitoreo del estado del dispositivo.

Estado: La propiedad de estado indica si el dispositivo está activo, inactivo o en algún otro estado relevante. Esto permite al sistema y a los usuarios tener una visibilidad clara sobre el funcionamiento de los dispositivos en tiempo real.

Configuraciones Específicas: Además del hardware ID y el estado, el **HomeDevice** puede incluir otras propiedades que se ajusten a las necesidades específicas del dispositivo. Por ejemplo, puede haber propiedades para la configuración de red, niveles de batería, o características personalizadas según el tipo de dispositivo (como temperatura para termostatos o niveles de luz para bombillas inteligentes).

Ventajas de Usar HomeDevices

La implementación de la clase HomeDevice ofrece varias ventajas:

Flexibilidad: Permite una gestión más flexible de los dispositivos dentro de un hogar, ya que cada dispositivo puede ser configurado de manera individual y asociado a las necesidades específicas del hogar.

Facilidad de Mantenimiento: Al tener una clase intermedia, el mantenimiento y la actualización de la información relacionada con los dispositivos se simplifica, facilitando tareas como la implementación de nuevos dispositivos o la modificación de los existentes.

Escalabilidad: La estructura permite escalar fácilmente el sistema a medida que se agregan más dispositivos o hogares, sin necesidad de complicar las relaciones entre las entidades.

La creación de la clase **MemberSettings** y para que funciona

La clase **MemberSettings** se creó para gestionar y personalizar las configuraciones de permisos de cada miembro dentro de un hogar. Su propósito es análogo al de los **Roles** con sus respectivas **PermissionKeys**, pero con un enfoque más centrado en las preferencias individuales de cada miembro.

Estructura de **MemberSettings**

Propiedades del Miembro: La clase **MemberSettings** contiene propiedades que representan las configuraciones específicas de un miembro. Esto puede incluir información sobre las preferencias de notificación, acceso a dispositivos y otros permisos relevantes.

Lista Virtual de Permissions: Al igual que en la clase **Role**, en **MemberSettings** hay una lista virtual de **Permission**. Esta propiedad permite que EF Core cargue automáticamente las claves de permisos asociadas al miembro desde la base de datos cuando se accede a ella. Esto proporciona flexibilidad y eficiencia al gestionar las configuraciones.

Funcionalidad de **MemberSettings**

La funcionalidad de **MemberSettings** se centra en permitir a cada owner personalizar su experiencia y establecer qué permisos desean activar. Algunas de sus características incluyen:

Permisos Personalizados: A través de la lista de **Permissions**, cada owner puede seleccionar permisos específicos que deseen habilitar o deshabilitar. Esto es particularmente útil en un entorno donde diferentes miembros pueden tener diferentes necesidades y niveles de acceso.

Preferencias de Notificación: La clase puede incluir configuraciones relacionadas con cómo y cuándo los miembros desean recibir notificaciones. Esto permite una gestión más centrada en el usuario, donde cada miembro puede ajustar sus preferencias en cuanto a las notificaciones.

Seguridad y Privacidad: Al permitir que cada miembro defina sus propios permisos, se mejora la seguridad y la privacidad dentro del hogar. Cada miembro tiene el control sobre qué información y funcionalidades son accesibles para ellos, lo que reduce el riesgo de acceso no autorizado.

Manejo de Excepciones y **Exceptionfilter**

En nuestro sistema, el manejo de excepciones se gestiona mediante un **ExceptionHandler** global que intercepta y procesa todas las excepciones lanzadas durante la ejecución de las peticiones. Este filtro no solo mejora la consistencia de las respuestas en caso de errores, sino que también permite una gestión centralizada de las excepciones, lo que genera que tengamos un código más controlado.

Excepciones Customizadas

Para lograr este control centralizado, utilizamos excepciones customizadas (o personalizadas). Estas excepciones son específicas del dominio de la aplicación y permiten manejar errores con mayor precisión, en lugar de depender de las excepciones genéricas del sistema. Cada excepción customizada representa un tipo particular de error, lo que facilita su identificación y tratamiento.

Funcionamiento del ExceptionFilter Global

El ExceptionFilter global se encarga de "atrapar" o cachear las excepciones que se lanzan durante el procesamiento de las solicitudes. Dependiendo del tipo de excepción que cachea, el filtro devuelve diferentes códigos de respuesta HTTP, permitiendo al cliente saber qué tipo de error ocurrió. Los códigos de respuesta que generalmente utilizamos incluyen:

- 400 Bad Request: Para errores de validación u otros problemas con la solicitud enviada por el cliente.
- 409 Conflict: Para situaciones en las que hay un conflicto con el estado actual del recurso (por ejemplo, intentos de crear un recurso que ya existe).

Hacer DeviceTypes Enum

Con el fin de optimizar el desarrollo y ahorrar tiempo, hemos decidido utilizar un enum llamado **DeviceTypes** para representar los diferentes tipos de dispositivos en nuestro sistema. Por el momento, este enum contiene dos tipos distintos: Sensor y Camera.

Este enfoque no solo simplifica la gestión de los tipos de dispositivos, sino que también ofrece una manera clara y eficiente de discriminar cómo se guardan estos dispositivos en la base de datos.

Implementación de la Seed Data

En la Seed Data se utiliza el método de modelBuilder llamado HasData para crear un usuario de rol admin que es todo lo necesario para empezar con la aplicación.

```
modelBuilder.Entity<User>().HasData(  
    new User  
    {  
        Id = Guid.Parse("b4a6e6cd-856e-4ad1-a87e-9f1b24d40a74"),  
        Name = "Admin",  
        LastName = "Admin",  
        Email = "admin@admin.com",  
        Password = "admin",  
        RoleID = adminRoleId  
    }  
);
```

Además se implementa todos los permisos tanto para los roles como los permisos predefinidos para los usuarios de una Home ya vienen integrados utilizando este mismo sistema.